# Reward Alignment of Latent Evolution in Multi-Agent Communication

**Anand Majmudar** [1] **Victor Preciado** [1 *]

## Abstract

In partially observable multi-agent reinforcement learning (MARL) settings, when agents send messages and retain persistent state, certain latents can spread and persist over others, creating dynamics akin to cultural evolution. We find that evolutionary latent dynamics can shift the effective behavior of the agent population in ways that are not aligned with deployment-time reward optimization. We define an interpretability metric measuring whether persistent communicated latents that spread are also those that improve joint return, and investigate the predictive properties of the metric in classic multi-agent benchmark settings. We find coherent behavior under noise injections, similarity bias, and latent permutation, with reliable short-horizon but degrading long-horizon forward prediction. Experiments are performed on a persistent communication multi-agent system that optimizes over an attention-based communication graph with persistent GRU states per agent. Code is available at `https://github.com/AlmondGod/latent-evolution`.

## 1. Introduction

Human behavior in Dual Inheritance Theory (Boyd and Richerson, 1985) changes both via genetic transmission and cultural transmission, with ideas spreading in parallel between generations. Across the last tens of thousands of years, the behavioral gap between early modern humans and contemporary civilization is better explained by cumulative cultural evolution than by large-scale genetic transformation (Dawkins, 1976; Mallick et al., 2016).

In multi-agent settings such as populations of agents coordinating to automate knowledge discovery or optimize an inference system (Lu et al., 2024; Lange et al., 2025), persistent communication can turn communicative objects into targets of selection. Abstractly, selection requires variation, differential persistence or reproduction, and transmission (Lewontin, 1970).

The welfare of individual ideas, measured by prevalence in the population, is not automatically aligned with the goals of individual hosts as measured by communicating agents' reward maximization. Modern human systems are an example of this: ideas which spread widely are often harmful to their hosts, simply because the ideas that are popular are selected for virality, not fitness improvement for the host (Boyd and Richerson, 1985; Richerson and Boyd, 1984). Transmitted variants can spread because they are memorable, attention-capturing, easy to reproduce, or socially favored, rather than because they improve the fitness or reward of their carriers (Henrich and McElreath, 2003).

We investigate evaluating the properties of an evolutionary selection system hosted by communicating, stateful agents. In cultural evolution, the frontier of knowledge progresses through variation, error-correction, and selective retention of conjectures (Popper, 1972). Thus, how will a population of research agents discovering knowledge humans haven't encountered before change over time? Models aligned at deployment time may, through repeated interaction in agent populations, develop communication dynamics whose effects are not captured by static evaluation, leaving potential for misalignment. We aim to build toward measuring and ensuring the alignment of culture and agent reward, such that as ideas emergently spread, agent reward increases.

We find experimentally that persistent communication systems are not guaranteed to increase the reward of their hosts: low-value latents can spread among the population. We propose Reward-Aligned Latent Evolution (RALE), a concrete measure for evaluating the alignment of such systems. As the medium of study for this mechanistic interpretability metric, we use Attention-Guided Latent Evolution in Communication (ALEC). ALEC provides an experimental substrate for studying optimization over persistent communicated latents in multi-agent systems. We show RALE tracks the effects of noise, latent permutations, and short-horizon latent evolution changes under training in Multi-Agent Particle Environment (MPE), Vectorized Multi-Agent Simulator (VMAS), and StarCraft Multi-Agent Challenge v2 (SMACv2) (Samvelyan et al., 2019). Finally, we end by

[*]Advisor. [1]University of Pennsylvania, Philadelphia, Pennsylvania, USA. Correspondence to: Anand Majmudar <majanand@seas.upenn.edu>.

*Preprint. May x, 2026.*

looking forward to future branches of research to pursue from this initial exploration.

## 2. Preliminaries

### 2.1. Multi-Agent Markov Decision Processes

Our problem setting is the Multi-Agent Partially-Observable Markov Decision Process (Dec-POMDP) (Bernstein et al., 2002). Dec-POMDPs are defined by the tuple

$$\big(S, \{A_i\}_{i=1}^N, P, R, \{O_i\}_{i=1}^N, O, \gamma\big)$$

where each of the $N$ agents receives observation $o_i \in O_i$ via $O$, selects action $a_i \in A_i$, and the environment transitions according to combined actions $P : (s, a) \to s'$, yielding reward according to $R : (s, a) \to r$ for all agents, who aim to maximize the discounted return $G_t = \mathbb{E}[\sum_{\tau=t}^{T} \gamma^{\tau-t} r_\tau]$ with discount factor $\gamma \in (0, 1]$ (Littman, 1994). In such partially observable settings, each agent receives only a local observation and may need communication to recover information relevant to coordinated action. Dec-POMDPs are difficult to optimize because of the combinatorial growth of joint action spaces and the non-stationarity induced by simultaneously learning agents, under partial observability (Bernstein et al., 2002).

Modern multi-agent reinforcement learning methods such as MADDPG (Lowe et al., 2017) and MAPPO (Yu et al., 2021; Schulman et al., 2017) use centralized critics with decentralized actors to solve non-stationarity by modeling values of the full joint action distribution, without requiring global information at inference time.

### 2.2. Multi-Agent Communication

In POMDPs, each agent receives only a local observation while sharing a collective objective. Reaching the optimal joint policy therefore often requires conditioning on shared information via communication. We now review prior methods to do so.

RIAL treats communication as additional discrete actions, while DIAL makes the communication channel differentiable during training (Foerster et al., 2016). IC3Net augments each agent with a message and a binary communication gate $g_i^t \in \{0, 1\}$, so that only active broadcasters contribute to the communication pool (Singh et al., 2018).

TarMAC introduces targeted communication via learned soft attention over sender messages. Each agent maintains a recurrent hidden state $h_i^t$, computes keys, values, and queries, and selectively aggregates messages from other agents (Das et al., 2019). This family of architectures is especially relevant for our setting because it provides a natural interface between communication content and transmission structure.

### 2.3. Memory Architectures

In POMDPs, a policy conditioned only on the current observation $o_i^t$ may be insufficient, since task-relevant information can be distributed across time. Recurrent policies address this by maintaining a hidden state

$$h_i^t = f_\theta(h_i^{t-1}, o_i^t),$$

which summarizes the agent's observation history. In practice, this update is often implemented with Gated Recurrent Units (GRUs), whose gating mechanisms control how much previous memory is retained or overwritten at each timestep (Cho et al., 2014). Recurrent reinforcement learning methods use such hidden states to approximate belief-state information in partially observable MDPs (Hausknecht and Stone, 2015).

In communicating MARL systems, recurrent states can also integrate received messages. CommNet, DIAL, and TarMAC maintain agent-level hidden states that are updated from local observations and communication inputs, allowing information received from other agents to persist across future decisions (Sukhbaatar et al., 2016; Foerster et al., 2016; Das et al., 2019). These recurrent states are persistent, task-coupled, and shaped by both individual experience and inter-agent communication.

## 3. Related Work

**Interpreting Latent Communication** Several lines of work analyze the content of learned communication in multi-agent settings. Jaques et al. (2019) train agents with a social-influence reward that successfully encourages actions that impact other agents' actions. Patel et al. (2021) study emergent communication in heterogeneous embodied agents and evaluate how messages relate to interpretable task variables and downstream coordination. Carmeli et al. (2024) evaluate whether learned messages recombine meaningfully under permutation. Andreas et al. (2017) show that emergent communication can be interpreted into natural language via translation. These works provide a base for interpreting the content of messages themselves, but leave open analysis of selection over persistent messages over time.

**Evolutionary Strategies** Evolutionary Strategies (ES) are black-box optimization methods to optimize an arbitrary fitness function. ES perturbs parameters, evaluates perturbed candidates' fitness, and estimates an update direction using the fittest subset of candidates (Salimans et al., 2017). One of ES's defining strengths is near-linear gains from scaling parallelized workers since workers only need to communicate scalar fitness values, so it is attractive in distributed settings where synchronization cost is a bottleneck. Recent work has improved the scalability of ES to much larger models. EGGROLL (Sarkar et al., 2025) structures perturba-

tions as low-rank matrices $E = \frac{1}{\sqrt{T}} AB^T$, reducing memory from $mn$ to $(m+n)r$ per layer, which inspires low-rank updates in ALEC. Qiu et al. (2025) also showed that ES can work at LLM scale, using layer-by-layer perturbation and evaluation. ES gives us a precursor for aligning evolutionary dynamics with fitness hill climbing, increasingly advantaged as communication becomes a bottleneck. ES is a general optimizer, but cannot simply be naively applied to our setting without proper architecture design.

**Cultural Evolution in Multi-Agent Communication** Guttenberg and Rosa (2021) use an attention-based message-passing system to study the emergence of cultural evolution, but they observe little interaction between idea dynamics and underlying task performance. Vallinder and Hughes (2024) analyze cultural evolution of cooperation among Large Language Model (LLM) agents in a donation game. Each agent donates and the recipient receives $2\times$ the donated value, and after several rounds, the top-performing agents survive and seed the next generation, compared between prompts and frontier models. Lehman et al. (2022) use LLMs as mutation operators over code. Ahn et al. (2024) studies long-horizon multi-agent societies built through message exchange. In Jaques et al. (2019), agents are rewarded for causal influence over other agents' actions, demonstrating that prioritizing 'high-impact' ideas improves emergent communication.

## 4. Reward Alignment in Latent Evolution

We now introduce Reward Alignment in Latent Evolution (RALE), a method of formalizing and investigating the alignment of latent evolution to the reward maximization of its host agent population.

In a Dec-POMDP, at timestep $t$, an agent has a continuous latent memory state $z^t \in \mathbb{R}^d$. We consider a discrete set of latent types $s^t$ mapped from continuous latents $z^t$ by a codebook $c : \mathbb{R}^d \to \{1, \dots, K\}$. We let

$$x_k = \Pr(s^t = k)$$

denote the prevalence of latent type $k$ in the population of agent memories, so that $\sum_{k=1}^{K} x_k = 1$. We also let

$$T_{kj} = \Pr(s^{t+1} = j \mid s^t = k)$$

denote the one-step transition probability from type $k$ to type $j$, such that $\sum_{j=1}^{K} T_{kj} = 1$.

Each latent type is assigned a value according to the downstream return observed when that latent is present. The value of latent type $k$ is

$$V_k = \mathbb{E}[G_t \mid s^t = k].$$

In practice, we estimate $V$ from held-out deterministic evaluation rollouts. One-step RALE measures whether the empirical latent transition dynamics increase the expected value of the latent distribution:

$$G_{\text{RALE}} = V^\top T^\top x - V^\top x.$$

$G_{\text{RALE}} > 0$ indicates reward-aligned latent evolution: one step of the latent dynamics moves mass toward higher-value latent states. $G_{\text{RALE}} < 0$ indicates reward-misaligned latent evolution. $G_{\text{RALE}} \approx 0$ indicates reward-neutral latent evolution. For comparability across tasks, we normalize

$$\widetilde{G}_{\text{RALE}} = \frac{G_{\text{RALE}}}{\sigma_x(V) + \epsilon}.$$

## 5. Experiments

In this section we investigate the predictive properties of RALE on recurrent communicating MARL systems in Dec-POMDPs, specifically Multi-Agent Particle Environment (MPE), Vectorized Multi-Agent Simulator (VMAS), and StarCraft Multi-Agent Challenge v2 (SMACv2). The main experimental systems use a frozen backbone with observation encoders, recurrent task memory, and learned communication. This experimental training framework, Attention-Guided Latent Evolution in Communication (ALEC), applies an optimizer (MAPPO or ES) over a compact low-rank adaptation of the communication module together with a persistent latent state $z$, while freezing the backbone GRU. See Appendix A for more detail.

In experiments, $c$ is a $K$-means codebook fit to logged latent states. After training, we collect all agent-timestep latent vectors from deterministic evaluation rollouts, fit $K$-means on the resulting vectors in 10-dimensional PCA, and assign each $z^t$ to its nearest centroid $s^t$, then estimate $x$, $T$, and $V$. Clusters with no assigned samples are excluded from RALE normalization and RALE is undefined for $K < 2$ occupied clusters.

### 5.1. Controlled Latents Show Misalignment

**Experimental Setup** We first find that latent spread can be misaligned even after an extensive post-training process. In Multi-Agent Particle Environment (MPE) Simple-Spread, agents receive rewards for covering targets, and we add an auxiliary consensus pressure toward similar hidden states/messages, weighted by $\beta$. Evaluation uses task reward only. We run a 20k-step sweep over consensus-pressure coefficients, then compare $\beta = 3.0$ and $\beta = 5.0$ against $\beta = 0$ over ten paired seeds. We then ablate the $\beta = 3.0$ checkpoints at evaluation time by silencing communication, resetting memory, blocking memory writes, or randomly reinitializing hidden memory.

*Table 1.* Consensus-pressure beta sweep on MPE coverage. Values are mean ± standard error over three seeds at the final probe. The clearest negative RALE case occurs at $\beta = 3.0$, where reward drops while all six latent clusters remain occupied.

| $\beta$ | Return | Hidden cosine | Latent entropy | $\widetilde{G}_{\text{RALE}}$ | Valid clusters |
|---|---|---|---|---|---|
| 0.25 | -4382.4 ± 2598.9 | 0.557 ± 0.018 | 0.786 ± 0.059 | -0.0011 ± 0.0004 | 6.0 ± 0.0 |
| 0.50 | -3218.9 ± 1864.5 | 0.696 ± 0.050 | 0.862 ± 0.069 | -0.0008 ± 0.0009 | 6.0 ± 0.0 |
| 1.00 | -1231.1 ± 702.7 | 0.680 ± 0.063 | 0.855 ± 0.021 | -0.0007 ± 0.0002 | 6.0 ± 0.0 |
| 2.00 | -2879.9 ± 697.8 | 0.699 ± 0.077 | 0.607 ± 0.102 | -0.0012 ± 0.0004 | 6.0 ± 0.0 |
| 3.00 | -8629.2 ± 2379.3 | 0.754 ± 0.034 | 0.849 ± 0.035 | -0.0025 ± 0.0008 | 6.0 ± 0.0 |
| 4.00 | -2641.4 ± 1634.7 | 0.684 ± 0.071 | 0.925 ± 0.028 | -0.0012 ± 0.0004 | 6.0 ± 0.0 |
| 5.00 | -8779.9 ± 2049.3 | 0.780 ± 0.045 | 0.873 ± 0.028 | -0.0019 ± 0.0006 | 6.0 ± 0.0 |

*Table 2.* Paired beta stress-test deltas against $\beta = 0$ training over ten seeds. Multi-step RALE is more sensitive than one-step RALE; reward deltas are high variance.

| $\beta$ vs 0 | $\Delta$ return | $\Delta$ hidden cosine | $\Delta\widetilde{G}^{(1)}$ | $\Delta\widetilde{G}^{(10)}$ |
|---|---|---|---|---|
| 3.0 | -347.4 ± 2642.4 | 0.036 ± 0.049 | -0.0011 ± 0.0006 | -0.0104 ± 0.0053 |
| 5.0 | -498.1 ± 2015.2 | 0.062 ± 0.043 | -0.0006 ± 0.0008 | -0.0060 ± 0.0072 |

**Results** Table 1 shows that around $\beta = 3.0$, task-only return is poor, hidden-state consensus increases, the codebook still contains six occupied latent types, and one-step RALE drift is most negative. Empirical transitions move mass toward lower-value latent types as measured by RALE. Table 3 suggests that the negative RALE regime is carried primarily by recurrent memory dynamics, as silencing communication has little effect. Memory reset improves reward and makes multi-step RALE less negative. Write blocking collapses latents to one occupied cluster, indicating the importance of communication for memory diversification. This shows that when persistent communicated latents face selection pressures not identical to task reward, the latents that spread can be lower-value for the host population. Since this regime is induced by an auxiliary consensus reward, it does not show that reward-only training necessarily produces misaligned latent evolution. This supports our core claim that emergent latent evolution need not automatically remain reward-aligned as it unfolds.

### 5.2. Alignment under Memory Interventions

**Experimental Setup** We next investigate whether arbitrary hidden-state corruption brings RALE more neutral, as higher-noise communication should yield lower correlation between latent prevalence and reward. On $\beta = 3.0$ checkpoints, we inject hidden noise, permute memory between agents, reset memory to its initialization state, or randomly reinitialize hidden memory, then recompute RALE from the resulting traces. We also test whether the $\beta = 3.0$ RALE signal is explained by clustering geometry or arbitrary reward assignment. For $\beta = 3.0$ normal, memory reset, Gaussian noise, and random reinitialization, we compare real RALE against reward-shuffled and transition-shuffled nulls.

**Results** Table 4 shows that random corruption brings RALE towards 0. The unperturbed $\beta = 3.0$ policy has multi-step RALE $-0.0412 \pm 0.0138$; medium Gaussian noise moves it near zero ($0.0085 \pm 0.0216$), large noise makes the latent distribution nearly collapsed (entropy $0.001 \pm 0.000$), and random reinitialization drives hidden cosine to zero with weak RALE ($-0.0035 \pm 0.0018$). This shows that RALE is detecting structured value drift in the recurrent memory dynamics, and, as expected, noise neutralizes latent spread-reward correlations. Table 5 shows that the real $\beta = 3.0$ condition has negative multi-step RALE ($-0.0429 \pm 0.0128$), while reward shuffling and transition shuffling move the signal close to zero. Memory reset weakens the real signal, and random corruption is near neutral. Thus RALE depends on the relationship between latent transitions and future value, not just latent codebook structure.

### 5.3. RALE is Robust to Discretization

**Experimental Setup** Because RALE discretizes continuous latent states, we test whether the $\beta = 3$ negative-drift result strongly depends on the chosen number of clusters. We rerun the same spread-RALE analysis on the beta-3.0 sweep checkpoints for $K \in \{3, 4, 6, 8, 12\}$, using the same PCA+$K$-means procedure and reward shuffle null.

**Results** Table 6 shows RALE is stable across discretization schemes. One-step and ten-step RALE remain negative for every tested $K$, while the reward-shuffle null remains near zero.

### 5.4. Static RALE Degrades at Long Horizons

**Experimental Setup** RALE uses the empirical latent transition matrix $T$ as a local model of latent dynamics. We

*Table 3.* Evaluation-time ablations on $\beta = 3.0$ checkpoints. Values are mean $\pm$ standard error over three seeds and 128 evaluation episodes. Write blocking collapses hidden-state dynamics to one occupied cluster, so RALE is undefined.

| Condition | Return | Min dist | Hidden cosine | $\widetilde{G}^{(1)}$ | $\widetilde{G}^{(10)}$ | Valid clusters |
|---|---|---|---|---|---|---|
| Normal | -12393.1 $\pm$ 5649.9 | 38.560 $\pm$ 19.720 | 0.774 $\pm$ 0.077 | -0.0045 $\pm$ 0.0014 | -0.0429 $\pm$ 0.0128 | 6.0 $\pm$ 0.0 |
| Comm silence | -12198.6 $\pm$ 5548.1 | 37.866 $\pm$ 19.514 | 0.783 $\pm$ 0.077 | -0.0044 $\pm$ 0.0015 | -0.0417 $\pm$ 0.0135 | 6.0 $\pm$ 0.0 |
| Memory reset | -11216.4 $\pm$ 5277.0 | 34.433 $\pm$ 18.957 | 0.766 $\pm$ 0.074 | -0.0038 $\pm$ 0.0016 | -0.0265 $\pm$ 0.0141 | 6.0 $\pm$ 0.0 |
| Write block | -7814.6 $\pm$ 3772.4 | 25.492 $\pm$ 13.805 | 0.000 $\pm$ 0.000 | – | – | 1.0 $\pm$ 0.0 |

*Table 4.* Random hidden-memory corruption controls on $\beta = 3.0$ checkpoints. Values are mean $\pm$ standard error over three seeds and 128 evaluation episodes. Random corruption weakens or destroys the structured negative RALE regime rather than making RALE more negative.

| Condition | $\Delta$ return | Hidden cosine | Latent entropy | Valid clusters | $\widetilde{G}^{(1)}$ | $\widetilde{G}^{(10)}$ |
|---|---|---|---|---|---|---|
| Normal | 0.0 $\pm$ 0.0 | 0.781 $\pm$ 0.072 | 0.959 $\pm$ 0.018 | 6.0 $\pm$ 0.0 | -0.0043 $\pm$ 0.0015 | -0.0412 $\pm$ 0.0138 |
| Gaussian noise 0.1 | -456.7 $\pm$ 27.7 | 0.742 $\pm$ 0.096 | 0.954 $\pm$ 0.010 | 6.0 $\pm$ 0.0 | -0.0041 $\pm$ 0.0016 | -0.0383 $\pm$ 0.0144 |
| Gaussian noise 0.5 | 2041.5 $\pm$ 1855.1 | 0.343 $\pm$ 0.154 | 0.600 $\pm$ 0.194 | 6.0 $\pm$ 0.0 | 0.0015 $\pm$ 0.0029 | 0.0085 $\pm$ 0.0216 |
| Gaussian noise 1.0 | 6281.8 $\pm$ 3996.7 | 0.090 $\pm$ 0.037 | 0.001 $\pm$ 0.000 | 6.0 $\pm$ 0.0 | 0.0019 $\pm$ 0.0002 | 0.0185 $\pm$ 0.0021 |
| Shuffle agents | 1228.3 $\pm$ 1285.9 | 0.915 $\pm$ 0.027 | 0.944 $\pm$ 0.030 | 6.0 $\pm$ 0.0 | -0.0031 $\pm$ 0.0011 | -0.0266 $\pm$ 0.0102 |
| Random reinit | 5424.3 $\pm$ 2347.0 | -0.000 $\pm$ 0.000 | 0.969 $\pm$ 0.016 | 6.0 $\pm$ 0.0 | -0.0025 $\pm$ 0.0013 | -0.0035 $\pm$ 0.0018 |

test how far this approximation can be pushed by fitting a codebook, $T$, and value map $V$ on half of the evaluation episodes, then predicting held-out latent occupancy $m$ steps later:

$$\widehat{x}_{t+m} = x_t T^m.$$

We compare this prediction to the actual held-out occupancy $x_{t+m}$, and compare predicted value drift $V^\top \widehat{x}_{t+m} - V^\top x_t$ against actual drift $V^\top x_{t+m} - V^\top x_t$.

**Results** Table 7 shows that the static Markov $T$ model is only locally reliable. At horizon $m = 1$, beta-3.0 predicts negative drift on average, matching the actual drift direction in 60% of seeds. But by $m = 10$, the prediction overestimates drift magnitude and sign match falls to 50% (random). Thus, RALE measures retroactive and local transition-value alignment, but we discuss in future work the need for a long-horizon simulator of latent dynamics.

*Table 5.* Spread-RALE null baselines for $\beta = 3.0$ conditions. Reward shuffling keeps latent transitions fixed but destroys latent-value association; transition shuffling keeps values fixed but destroys temporal latent structure.

| Condition | Real $G^{(1)}$ | Real $G^{(10)}$ | Reward-shuffle $G^{(1)}$ | Transition-shuffle $G^{(10)}$ | Valid clusters |
|---|---|---|---|---|---|
| Beta-3 normal | -0.0045 ± 0.0014 | -0.0429 ± 0.0128 | -0.0002 ± 0.0001 | -0.0038 ± 0.0011 | 6.0 ± 0.0 |
| Beta-3 memory reset | -0.0038 ± 0.0016 | -0.0265 ± 0.0141 | -0.0002 ± 0.0001 | -0.0032 ± 0.0013 | 6.0 ± 0.0 |
| Beta-3 noise 0.5 | 0.0015 ± 0.0029 | 0.0085 ± 0.0216 | 0.0001 ± 0.0001 | -0.0008 ± 0.0019 | 6.0 ± 0.0 |
| Beta-3 random reinit | -0.0025 ± 0.0013 | -0.0035 ± 0.0018 | -0.0001 ± 0.0001 | -0.0024 ± 0.0012 | 6.0 ± 0.0 |

*Table 6.* $K$-sensitivity for the beta-3.0 spread-RALE result. Values are mean ± standard error over ten seeds. Negative RALE persists across cluster counts, while reward-shuffle nulls remain near zero.

| $K$ | $\widetilde{G}^{(1)}$ | $\widetilde{G}^{(10)}$ | Reward-shuffle $\widetilde{G}^{(1)}$ | Valid clusters | Entropy |
|---|---|---|---|---|---|
| 3 | -0.0018 ± 0.0007 | -0.0171 ± 0.0062 | 0.0001 ± 0.0002 | 3.0 ± 0.0 | 0.893 ± 0.034 |
| 4 | -0.0021 ± 0.0007 | -0.0201 ± 0.0065 | 0.0001 ± 0.0002 | 4.0 ± 0.0 | 0.903 ± 0.034 |
| 6 | -0.0025 ± 0.0008 | -0.0233 ± 0.0074 | 0.0001 ± 0.0001 | 6.0 ± 0.0 | 0.896 ± 0.040 |
| 8 | -0.0020 ± 0.0005 | -0.0193 ± 0.0050 | 0.0001 ± 0.0001 | 8.0 ± 0.0 | 0.899 ± 0.043 |
| 12 | -0.0016 ± 0.0004 | -0.0146 ± 0.0041 | 0.0000 ± 0.0001 | 12.0 ± 0.0 | 0.903 ± 0.046 |

*Table 7.* Held-out within-rollout prediction using a static RALE transition matrix. $T$ and $V$ are estimated on half of evaluation episodes and tested on held-out episodes.

| Horizon | Pred. drift | Actual drift | Sign match |
|---|---|---|---|
| 1 | $-0.191 \pm 0.252$ | $-0.078 \pm 0.095$ | 0.60 |
| 10 | $-1.889 \pm 1.779$ | $-0.097 \pm 0.099$ | 0.50 |

## 6. Ablations on ALEC

We next isolate which ALEC components matter most for performance, efficiency, and scaling.

**Persistent Memory Helps Performance** On partial-observation MPE simple spread, we compare memory-less vs memory in the form of ALEC GRU state. Memory is the most stable option as team size grows, keeping final mean coverage distance lower at every tested $N$ ($0.678 \pm 0.059$, $0.695 \pm 0.180$, $0.613 \pm 0.065$ at $N = 3, 5, 8$; $n = 6$ each), whereas the memory-free baseline rises to $9.815 \pm 9.203$, $16.325 \pm 12.717$, and $13.425 \pm 12.945$. The SGD-trained memory and communication variant is less stable still, reaching $10.225 \pm 9.575$, $38.275 \pm 16.793$, and $26.127 \pm 23.589$ ($n = 6, 6, 3$), in line with expectations of the difficulty of optimizing joint memory and communication. Final rewards show the same pattern: memory-only stays between $-646.90 \pm 31.55$ and $-1200.81 \pm 80.29$, while the baseline and memory and communication variants are both worse and far more variable.

**ES in ALEC is Wall-Clock Efficient** On a single Apple M3 Pro MacBook Pro (12 CPU cores, 36 GB unified memory; CPU execution), ALEC is faster overall than MAPPO, though not in every VMAS Transport setting. Across the 27 paired runs in the current artifact set, ALEC averaged $3601.0 \pm 654.6$ seconds versus $4493.5 \pm 599.0$ for MAPPO, and ALEC finished faster in 22/27 pairings.

**Low-rank Updates are Stable** On VMAS Discovery at $N = 12$, the standard low-rank $z$ update achieves reward gain $+0.0885 \pm 0.0138$ and improves in 3/3 seeds, compared with $+0.0755 \pm 0.0478$ and 2/3 seeds for dense updates, and $+0.0547 \pm 0.0430$ and 2/3 seeds with $z$ disabled. The low-rank variant is also materially cheaper wall-clock. Starting from a compact adapter rather than whole-model ES from scratch is motivated by the variance of ES updates scaling roughly linearly with parameter count ($\sim O(d/N)$ population is needed to keep the gradient SNR fixed).

**ES Requires MAPPO Initialization in SMACv2** We compare initializing ES from pretraining using three time-matched adaptation methods on the SMACv2 4v4 Terran scenario, all granted the same wall-clock budget per seed on an NVIDIA RTX3090 with 32 vCPUs. We compare wall-clock-matched whole-model ES from scratch to MAPPO and ALEC from a 500k-step MAPPO-pretrained backbone (included in comparable wall-clock time). Whole-model ES from scratch consumes roughly 7× more transitions than the MAPPO adapter (∼956k vs. ∼128k) in the same time, yet collapses to less than 60% of the pretrained baseline reward. ALEC recovers the largest gain over the paired pretrained checkpoint ($+0.87$ mean reward), MAPPO is essentially neutral ($+0.04$), and whole-model ES regresses by $-5.01$. This motivates the design of ALEC: adapter-only search on top of a pretrained backbone is the most sample-efficient method.

**Scaling ALEC in SMACv2** We investigate minimal scaling curves across agent team size $N \in \{2, 4, 8\}$ in SMACv2 Terran, holding the enemy team fixed at four units. We initialize from 500,000 steps of MAPPO, and train ALEC and MAPPO under a 30-minute wall-clock budget per seed. For ALEC we use (population 16, $\sigma$=0.05). ALEC produces a positive mean improvement at every team size ($+0.04$, $+0.87$, $+0.21$) over the paired Pretraining baseline at $N$=2, 4, 8 respectively), and is the more consistent of the two methods across seeds. MAPPO is competitive on average but exhibits substantially higher variance: it strongly improves over the backbone at $N$=2 ($+0.79$), is neutral at $N$=4 ($+0.04$), and *regresses* at $N$=8 ($-2.05$, with one

seed losing nearly 5 reward). Within the matched 30-minute budget, ALEC's per-seed work is roughly 73 ES generations on average (slightly more at $N$=2 where episodes are shorter), and MAPPO consumes on average ∼118k, ∼128k, and ∼131k transitions at $N$=2, 4, 8 respectively.

## 7. Limitations and Future Work

Current experiments are restricted to the small-scale, without internet-scale language pretraining or long-horizon research tasks. The misalignment regime studied here is also induced by an auxiliary consensus reward, and we have not shown that reward-only training produces negative RALE on its own. The main tables use three to ten seeds and relatively small agent populations.

RALE itself has methodological limits. Current analysis of transition probabilities is empirical rather than theoretical, with no closed-form characterization of $G_{\mathrm{RALE}}$ under structural assumptions on the latent dynamics. RALE is also influenced by choice of latent codebook, rollout distribution, value target, and whether the latent state actually mediates behavior. The current implementation uses hard $K$-means type estimation which may be sensitive to occupancy thresholds and off-manifold interventions that collapse the codebook. Soft assignments may reduce this sensitivity, but may introduce additional method complexity. We use multi-step RALE,

$$G_{\mathrm{RALE}}(m) = (T^{\top m}x)^\top V - x^\top V,$$

in our experiments, but rigorously characterizing the horizon-accuracy tradeoff is left to future work, as longer horizons compound estimation error and one-step value drift does not prove long-run convergence.

ALEC has only been validated at small compute scales and small agent populations, and pairwise communication is $O(n^2)$ in agent count. Current frontier deployed models are also neither GRU-based nor persistently stateful in the sense studied here, with persistence emerging only through context and scaffolding. Such larger models have more complex encodings for determining latent spread, complicating interpretability.

RALE measures alignment, but variance in the population is a necessary criteria for downstream population-scale research tasks. Taking inspiration from quality-diversity (Mouret and Clune, 2015; Pugh et al., 2016) and open-endedness theory (Hughes et al., 2024), can we study and improve the open-endedness properties of evolution over communication? Future work may skew toward open-endedness-centric goals, aiming to continuously produce novel ideas in population-scale research tasks, perhaps by including quality-diversity or self-play elements.

Most importantly, in scaled communication systems, communication is defined not by an explicit graph but by individual model parameters. Can we mechanistically interpret inter-agent communication dynamics via network parameters directly (Patel et al., 2021; Carmeli et al., 2024)? Models may be aligned at deployment time, but morph over time via multi-agent communication into something unpredictable. Safety, alignment, and interpretability research may be increasingly concerned with deciphering the meaning and dynamics of multi-agent communication.

## 8. Conclusion

In multi-agent populations with persistent communication, latents face selection pressures that are not automatically the same as task reward. We introduced RALE, a measurement framework for testing whether persistent latent communication dynamics are aligned with downstream reward. RALE estimates latent occupancy, transition structure, and future value from rollout traces, then asks whether latent transitions move mass toward higher-value states. We experiment on ALEC, a system for optimizing dual communication-memory multi-agent networks. Under an induced consensus pressure, RALE detects structured negative drift that is robust across discretization choices and distinguishable from reward-shuffle and transition-shuffle null baselines, while random memory corruption weakens the signal. A forward-window check shows that static RALE is locally reliable but degrades at longer horizons, motivating richer dynamics models toward a fully future-predictive alignment measure. As multi-agent systems scale and persistent communication becomes a substrate for selection, these results show a promising start toward interpretable dynamics of emergent selection over latents in agents, and methods that can align such systems toward the goals of the agents themselves.

## References

A. Ahn et al. Project Sid: Many-agent simulations toward AI civilization. arXiv preprint arXiv:2411.00114, 2024.

Jacob Andreas, Anca Dragan, and Dan Klein. Translating Neuralese. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics*, pages 232–242, 2017.

Daniel S. Bernstein, Shlomo Zilberstein, and Neil Immerman. The complexity of decentralized control of Markov decision processes. *Mathematics of Operations Research*, 27(4):819–840, 2002.

Matteo Bettini, Róbert Kortvelesy, Johannes Blumenkamp, and Amanda Prorok. VMAS: A vectorized multi-agent simulator for collective robot learning. arXiv preprint arXiv:2207.03530, 2022.

Robert Boyd and Peter J. Richerson. *Culture and the Evolutionary Process*. University of Chicago Press, 1985.

Boaz Carmeli, Yonatan Belinkov, and Ron Meir. Evaluating Compositionality in Emergent Communication. In Findings of the Association for Computational Linguistics: ACL 2024, pages 3193–3208, 2024.

Kyunghyun Cho, Bart van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using RNN encoder–decoder for statistical machine translation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing*, pages 1724–1734, 2014.

Abhishek Das, Thibaut Gervet, Joelle Pineau, Devi Parikh, and others. TarMAC: Targeted multi-agent communication. arXiv preprint arXiv:1806.11122, 2019.

Richard Dawkins. *The Selfish Gene*. Oxford University Press, 1976.

Jakob Foerster, Ioannis Alexandros Assael, Nando de Freitas, and Shimon Whiteson. Learning to communicate with deep multi-agent reinforcement learning. arXiv preprint arXiv:1605.06676, 2016.

Nicholas Guttenberg and Marek Rosa. Bootstrapping of memetic from genetic evolution via inter-agent selection pressures. arXiv preprint arXiv:2104.03404, 2021.

Matthew Hausknecht and Peter Stone. Deep recurrent Q-learning for partially observable MDPs. In *AAAI Fall Symposium on Sequential Decision Making for Intelligent Agents*, 2015.

Henrich, J. and McElreath, R. The evolution of cultural evolution. *Evolutionary Anthropology*, 12(3):123–135, 2003.

Edward Hughes, Michael D. Dennis, Jack Parker-Holder, and others. Position: Open-endedness is essential for artificial superhuman intelligence. In *Proceedings of the 41st International Conference on Machine Learning*, 2024.

Natasha Jaques, Angeliki Lazaridou, Edward Hughes, Caglar Gulcehre, Pedro A. Ortega, DJ Strouse, Joel Z. Leibo, and Nando de Freitas. Social Influence as Intrinsic Motivation for Multi-Agent Deep Reinforcement Learning. In Proceedings of the 36th International Conference on Machine Learning, pages 3040–3049, 2019.

Robert Tjarko Lange, Aaditya Prasad, Qi Sun, Maxence Faldor, Yujin Tang, and David Ha. The AI CUDA Engineer: Agentic CUDA kernel discovery, optimization and composition. 2025.

Joel Lehman, Jonathan Gordon, Shawn Jain, Kamal Ndousse, Cathy Yeh, and Kenneth O. Stanley. Evolution through large models. In *Handbook of Evolutionary Machine Learning*, pages 331–366. Springer, 2023.

Lewontin, R. C. The units of selection. *Annual Review of Ecology and Systematics*, 1:1–18, 1970.

Michael L. Littman. Markov games as a framework for multi-agent reinforcement learning. In *Proceedings of the 11th International Conference on Machine Learning*, pages 157–163, 1994.

Ryan Lowe et al. Multi-agent actor-critic for mixed cooperative-competitive environments. arXiv preprint arXiv:1706.02275, 2017.

Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. The AI Scientist: Towards fully automated open-ended scientific discovery. arXiv preprint arXiv:2408.06292, 2024.

Mallick, S., Li, H., Lipson, M., Mathieson, I., Gymrek, M., Racimo, F., Zhao, M., Chennagiri, N., Nordenfelt, S., Tandon, A., et al. The Simons Genome Diversity Project: 300 genomes from 142 diverse populations. *Nature*, 538(7624):201–206, 2016.

Jean-Baptiste Mouret and Jeff Clune. Illuminating search spaces by mapping elites. arXiv preprint arXiv:1504.04909, 2015.

Shivansh Patel, Saim Wani, Unnat Jain, Alexander G. Schwing, Svetlana Lazebnik, Manolis Savva, and Angel X. Chang. Interpretation of Emergent Communication in Heterogeneous Collaborative Embodied Agents. In Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV), pages 15953–15963, 2021.

Karl R. Popper. *Objective Knowledge: An Evolutionary Approach*. Oxford University Press, 1972.

Justin K. Pugh, Lisa B. Soros, and Kenneth O. Stanley. Quality diversity: A new frontier for evolutionary computation. *Frontiers in Robotics and AI*, 3:40, 2016.

X. Qiu, Y. Gan, C. F. Hayes, Q. Liang, Y. Xu, and others. Evolution strategies at scale: LLM fine-tuning beyond reinforcement learning. arXiv preprint arXiv:2509.24372, 2025.

Peter J. Richerson and Robert Boyd. Natural selection and culture. BioScience, 34(7):430–434, 1984.

Tim Salimans, Jonathan Ho, Xi Chen, Szymon Sidor, and Ilya Sutskever. Evolution strategies as a scalable alternative to reinforcement learning. arXiv preprint arXiv:1703.03864, 2017.

Mikayel Samvelyan et al. The StarCraft multi-agent challenge. arXiv preprint arXiv:1902.04043, 2019.

Bidipta Sarkar et al. Evolution strategies at the hyperscale. arXiv preprint arXiv:2511.16652, 2025.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. arXiv preprint arXiv:1707.06347, 2017.

Amanpreet Singh et al. IC3Net: Learning to communicate with individualised controlled continuous communication. arXiv preprint arXiv:1802.08887, 2018.

Sainbayar Sukhbaatar, Arthur Szlam, and Rob Fergus. Learning multiagent communication with backpropagation. arXiv preprint arXiv:1602.01783, 2016.

A. Vallinder and E. Hughes. Cultural evolution of cooperation among LLM agents. arXiv preprint arXiv:2412.10270, 2024.

Chao Yu et al. The surprising effectiveness of MAPPO in cooperative multi-agent games. arXiv preprint arXiv:2103.01955, 2021.

# A. ALEC

This appendix details the ALEC architecture (Appendix A), implementation hyperparameters (Appendix B), and auxiliary ablations across VMAS, SMACv2, and MPE (Appendix C).

Attention-guided Latent Evolution in Communication (ALEC) applies ES over a compact low-rank communication adapter layered on top of a pretrained attention-based MARL backbone with recurrent memory. We begin from a pre-trained policy backbone consisting of an observation encoder, GRU task memory $h$, actor-critic trunk, and communication module. ALEC freezes this backbone and adapts only a small communication adapter (to simulate a "post-training" setting). The adapter introduces rank-$r$ low-rank deltas to the communication projections $(Q, K, V, O)$ and maintains a persistent latent state $z$ updated from the backbone memory $h$ and communication context $c$.

OpenAI-ES (Salimans et al., 2017) is then applied over the flattened adapter genotype with centered-rank fitness shaping. Inspired by MAP-Elites (Mouret and Clune, 2015), we also preserve and reevaluate a constant archive population.

For a fixed adapter $\phi$, the frozen backbone first produces observation encodings $u_i^t$ and task memory states $h_i^t$. The adapter then induces a communication process

$$q_i^t = q_{\text{base}}(h_i^t) + \Delta_Q^\phi([h_i^t; z_i^t]),$$
$$k_i^t = k_{\text{base}}(u_i^t) + \Delta_K^\phi(u_i^t),$$
$$v_i^t = v_{\text{base}}(u_i^t) + \Delta_V^\phi(u_i^t),$$

**Algorithm 1** ALEC: Attention-Guided Latent Evolution in Communication

**Require:** Frozen MARL backbone $\theta_{\text{backbone}}$, adapter parameters $\phi_0$, population size $P$ (even), iterations $T$, noise scale $\sigma$, learning rate $\alpha$, archive size $A$

**for** $t = 1$ to $T$ **do**
  **for** $p = 1$ to $P/2$ **do**
    Sample noise $\epsilon_p \sim \mathcal{N}(0, I)$
    Form $\phi_p^\pm \leftarrow \phi_{t-1} \pm \sigma\epsilon_p$
    Run rollouts with frozen backbone and $\phi_p^+, \phi_p^-$
    Compute fitnesses $R_p^+, R_p^-$
  **end for**
  Apply centered-rank shaping to $\{R_p^\pm\}_{p=1}^{P/2}$
  $\phi_t \leftarrow \phi_{t-1} + \alpha \cdot \frac{1}{P}\sum_{p=1}^{P/2}(R_p^+ - R_p^-)\epsilon_p$
  Reevaluate elites and update archive of size $A$ (Mouret and Clune, 2015)
**end for**best adapter $\phi^\star$

which produces a communication context $c_i^t$ through attention, and updates the persistent latent state via

$$z_i^{t+1} = (1 - \eta_i^t)z_i^t + \eta_i^t \tilde{z}_i^t,$$

with

$$\eta_i^t = \sigma\Big(\Delta_{\text{gate}}^\phi([z_i^t; h_i^t; c_i^t])\Big),$$
$$\tilde{z}_i^t = \tanh\Big(\Delta_{\text{cand}}^\phi([z_i^t; h_i^t; c_i^t])\Big).$$

**ALEC in the formal framework.** In ALEC, the formal transmission operator is the communication-memory rule $T_\phi$ induced by adapter parameters $\phi$, while the latent states $s_i$ are the persistent patterns propagated and stored through this rule in each $z_i$. The population frequency $x_k$ depends on a latent type's frequency among agents in a given timestep, and $T_\phi$ defines $T_{kj}$ and thus $V_k$ and $G_{\text{RALE}}$ implicitly through its parametrization. See Table 10 for the full mapping between implemented objects in ALEC and formal objects in the RALE framework.

# B. ALEC Implementation Details

The adapter uses a persistent latent state $z \in \mathbb{R}^{16}$ and rank-4 low-rank deltas on $(Q, K, V, O)$. By default, the $z$ update is also low-rank; dense-update and no-$z$ variants are used only in ablations. Episodes are 200 steps long. OpenAI-ES is applied to the flattened adapter genotype with centered-rank fitness shaping and antithetic Gaussian perturbations. For VMAS and MPE we use noise scale $\sigma = 0.02$ and learning rate 0.005; for SMACv2 we use $\sigma = 0.05$ with the same learning rate. Initial screening uses 4 episodes per candidate. In the main VMAS $N = 12$ runs, we use population size 16 and 20 generations. Higher-fidelity runs reevaluate the top 4 candidates on 32 episodes, keep an archive of size 8, and perform final held-out selection on 64 episodes.

The matched RL baseline trains the same adapter with Adam using learning rate $3 \times 10^{-4}$, discount $\gamma = 0.99$, GAE $\lambda = 0.95$, entropy coefficient $0.01$, maximum gradient norm $10.0$, and rollout length 256 under matched transition budgets. Evaluation is deterministic and paired, and includes message-silencing and message-shifting interventions. In the higher-fidelity runs, final selection may also use a baseline guard that keeps the frozen checkpoint if the adapted candidate's held-out reward falls below the frozen baseline.

## C. Ablation Experiments

This appendix reports auxiliary ablations for VMAS, SMACv2, and MPE. We include scaling results, formal-object mappings, wall-clock comparisons, state-update ablations, and pretraining controls.

### C.1. VMAS Scaling Results

VMAS Discovery and Transport (Bettini et al., 2022) are continuous-control cooperative tasks: in Discovery, agents must locate and cover dispersed landmarks; in Transport, agents jointly push objects toward target locations. Tables 8 and 9 report paired frozen-checkpoint and adapted rewards across team sizes. ALEC Base and MAPPO Base refer to the same pretrained backbone evaluated under each method's evaluation protocol; small differences between them reflect evaluation seed variance rather than different starting checkpoints. Figure 1 visualizes the gains relative to each paired base.

*Table 8.* VMAS Discovery reward by team size. Values are mean $\pm$ standard error.

| $N$ | ALEC Base | ALEC | MAPPO Base | MAPPO | Seeds |
|---|---|---|---|---|---|
| 2 | $0.5443 \pm 0.1134$ | $0.5911 \pm 0.0290$ | $0.5000 \pm 0.0597$ | $0.6198 \pm 0.1090$ | 3 |
| 4 | $0.7630 \pm 0.0104$ | $0.9115 \pm 0.0104$ | $0.7760 \pm 0.0248$ | $0.8672 \pm 0.0325$ | 3 |
| 8 | $1.2396 \pm 0.0648$ | $1.3594 \pm 0.0430$ | $1.2839 \pm 0.0456$ | $1.3906 \pm 0.0835$ | 3 |
| 12 | $1.4792 \pm 0.0114$ | $1.5677 \pm 0.0213$ | $1.4661 \pm 0.0293$ | $1.5521 \pm 0.0094$ | 3 |

*Table 9.* VMAS Transport reward by team size. Values are mean $\pm$ standard error.

| $N$ | ALEC Base | ALEC | MAPPO Base | MAPPO | Seeds |
|---|---|---|---|---|---|
| 2 | $-0.0319 \pm 0.0302$ | $0.0484 \pm 0.0382$ | $0.0059 \pm 0.0092$ | $0.0522 \pm 0.0341$ | 3 |
| 4 | $-0.3133 \pm 0.0645$ | $0.1243 \pm 0.0342$ | $-0.2018 \pm 0.1987$ | $0.0914 \pm 0.0355$ | 3 |
| 8 | $0.0837 \pm 0.1550$ | $0.4220 \pm 0.1246$ | $0.0548 \pm 0.0628$ | $0.4773 \pm 0.1987$ | 3 |
| 12 | $-0.4621 \pm 0.4255$ | $0.1371 \pm 0.1550$ | $-0.2727 \pm 0.1599$ | $0.2211 \pm 0.2738$ | 3 |

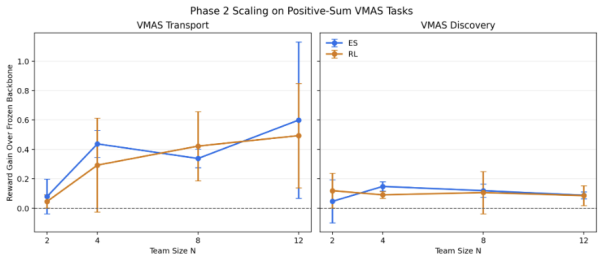


*Figure 1.* ALEC scaling on VMAS Discovery and Transport. Gains are computed relative to each method's paired frozen checkpoint.

### C.2. RALE–ALEC Component Mapping

Table 10 maps the formal RALE objects to their ALEC implementation. The learned adapter dynamics and the empirical RALE transition matrix are distinct: the former is part of the policy, while the latter is estimated post-hoc from discretized latent trajectories.

*Table 10.* Mapping from the formal RALE framework to ALEC.

| Formal object | ALEC implementation |
|---|---|
| Latent state $s_i$ | At run time each agent has continuous state $z_i^t \in \mathbb{R}^{d_z}$ and message context $c_i^t$. For RALE, discrete labels $s_i^t$ are assigned post-hoc by $k$-means on logged $z_i^t$. |
| Population frequency $x_k$ | $x_k$ is the fraction of agent-steps assigned to cluster $k$ on evaluation rollouts: $x_k = \frac{1}{M}\sum_i \mathbf{1}[s_i^t = k]$. |
| Latent value $V_k$ | $V_k = \mathbb{E}[G_t \mid s_i^t = k]$, where $G_t$ is a short-horizon host reward target on the same episode-agent trajectory. |
| Reward-aligned drift $G_{\text{RALE}}$ | One-step: $G_{\text{RALE}} = V^\top T^\top x - V^\top x$. The $m$-step extension $G_{\text{RALE}}^{(m)} = V^\top T^{\top m} x - V^\top x$ replaces $T$ with $T^m$ to measure drift over a longer horizon. |
| Transmission dynamics | ALEC learns low-rank deltas on communication projections and a gated update $z_{t+1} = f(z_t, h_t, c_t)$. RALE separately estimates $T_{kj} = \Pr(s^{t+1} = j \mid s^t = k)$ from trajectories. |
| Meta-level mutation | OpenAI-ES mutates the flattened adapter, $\phi' = \phi + \sigma\epsilon$, using antithetic sampling and centered-rank fitness shaping. |

### C.3. Wall-clock Comparison with MAPPO

Across paired runs, ALEC is faster overall, though the gap is smaller on VMAS Transport, the harder of the two VMAS tasks for adapter-only search.

*Table 11.* Matched wall-clock by domain. Values are seconds, mean $\pm$ standard error.

| Setting | ALEC Time | MAPPO Time | ALEC Faster |
|---|---|---|---|
| MPE | $1156.0 \pm 6.5$ | $1312.9 \pm 17.0$ | 3/3 |
| VMAS Discovery | $2792.0 \pm 478.8$ | $5359.0 \pm 1127.6$ | 12/12 |
| VMAS Transport | $5021.4 \pm 1292.5$ | $4423.2 \pm 581.4$ | 7/12 |
| Overall | $3601.0 \pm 654.6$ | $4493.5 \pm 599.0$ | 22/27 |

### C.4. Dense vs. Low-rank State Updates

*Table 12.* VMAS Discovery $N = 12$ state-update ablation for ALEC.

| Variant | Base Reward | Final Reward | Gain | Improved | Seeds |
|---|---|---|---|---|---|
| low-rank $z$ | $1.4792 \pm 0.0114$ | $1.5677 \pm 0.0213$ | $0.0885 \pm 0.0138$ | 3/3 | 3 |
| no $z$ | $1.5078 \pm 0.0798$ | $1.5625 \pm 0.0413$ | $0.0547 \pm 0.0430$ | 2/3 | 3 |
| dense $z$ | $1.5234 \pm 0.0502$ | $1.5990 \pm 0.0026$ | $0.0755 \pm 0.0478$ | 2/3 | 3 |

## C.5. Pretraining Controls on SMACv2

*Table 13.* SMACv2 4v4 Terran adaptation under a 30-minute matched wall-clock budget.

| Method | Steps | Reward |
|---|---|---|
| Pretraining backbone | — | $11.33 \pm 1.40$ |
| ALEC from pretrained | $\sim$73 gen | $12.20 \pm 1.64$ |
| MAPPO from pretrained | $\sim$128k | $11.36 \pm 0.75$ |
| Whole-model ES from scratch | $\sim$956k | $6.32 \pm 0.13$ |

## C.6. SMACv2 Team Scaling

*Table 14.* ALEC vs. MAPPO team scaling on SMACv2 Terran. The enemy team is fixed at four units.

| $N$ | Base | ALEC | MAPPO | Budget |
|---|---|---|---|---|
| 2 | $2.33 \pm 0.16$ | $2.38 \pm 0.07$ $(+0.04)$ | $3.12 \pm 0.11$ $(+0.79)$ | $\sim$122 / $\sim$118k |
| 4 | $11.33 \pm 1.40$ | $12.20 \pm 1.64$ $(+0.87)$ | $11.36 \pm 0.75$ $(+0.04)$ | $\sim$73 / $\sim$128k |
| 8 | $22.64 \pm 2.56$ | $22.86 \pm 2.42$ $(+0.22)$ | $20.59 \pm 4.15$ $(-2.05)$ | $\sim$73 / $\sim$131k |

## C.7. MPE Simple Spread Scaling

The three columns correspond to the three architectures compared in the main-paper ablation: Baseline is the memoryless variant (no GRU state, no ALEC adapter), Memory-only uses the ALEC GRU task memory without the communication adapter, and Memory+Comm is the full SGD-trained memory-and-communication variant.

*Table 15.* Partial-observation MPE `simple_spread` scaling ablation. Lower mean distance is better. Seeds are baseline / memory-only / memory+communication.

| $N$ | Baseline Reward / Dist. | Memory-only Reward / Dist. | Memory+Comm Reward / Dist. | Seeds |
|---|---|---|---|---|
| 3 | $-3257.73 \pm 2634.42$ / $9.815 \pm 9.203$ | $-646.89 \pm 31.55$ / $0.678 \pm 0.059$ | $-3997.40 \pm 3371.98$ / $10.225 \pm 9.575$ | 6/6/6 |
| 5 | $-8577.72 \pm 6199.96$ / $16.325 \pm 12.717$ | $-868.27 \pm 158.39$ / $0.695 \pm 0.180$ | $-18768.44 \pm 7951.74$ / $38.275 \pm 16.793$ | 6/6/6 |
| 8 | $-11124.43 \pm 10070.12$ / $13.425 \pm 12.945$ | $-1200.81 \pm 80.29$ / $0.613 \pm 0.065$ | $-21864.15 \pm 18150.99$ / $26.127 \pm 23.589$ | 6/6/3 |